

Herança e polimorfismo

Você já sabe criar classes e objetos. Agora vamos fazer uma classe aproveitar outra, e o seu RPG virar um jogo com personagens diferentes de verdade.

Você trabalha no seu próprio ritmo.

Cada capítulo termina em um checkpoint com um teste.

Passou no teste, siga em frente.

ANTES DE COMEÇAR

Como usar este material

Este documento é para você seguir sozinho. Leia, escreva o código, rode, confira o resultado com o que está na página. Quando travar, chame o professor.

De onde você parte

Você vai usar os dois arquivos que a turma já escreveu no Dia 5:

ARQUIVO	O QUE TEM DENTRO
<code>personagem.py</code>	A classe <code>Personagem</code> , com <code>atacar</code> , <code>defender</code> , <code>receber_dano</code> e <code>usar_pocao</code> .
<code>main.py</code>	O laço de combate e a lista de inimigos.

Não apague nada. Durante quase todo este documento, o `personagem.py` fica **exatamente como está**. Isso não é preguiça: faz parte da lição. Uma classe bem escrita consegue ser aproveitada por outras sem precisar mudar.

O caminho completo

1 O problema

Você tenta criar dois personagens novos só com o que já sabe. Vai dar trabalho, e é de propósito.

2 Herança

Uma classe aproveita outra. Aparece o `super()`.

3 Sobrescrever métodos

Trocar o comportamento da classe-mãe, ou acrescentar a ele.

4 Polimorfismo

O laço de combate funciona com qualquer personagem, sem saber qual é.

5 Fechamento

O que você construiu e o que falta.

→ Parte 2 (outro PDF)

Encapsulamento, seleção de personagem, chefe final e o jogo fechado.

Os quatro tipos de caixa

CAIXA	O QUE FAZER
Checkpoint	Marca o fim de uma etapa. Diz o que o seu código precisa fazer nesse ponto.
Teste	Um código curto para você rodar. Se o resultado bater, você entendeu. Se não bater, a caixa diz o que provavelmente aconteceu.
Leia a fonte	Uma pergunta cuja resposta está na documentação oficial do Python. Procure e responda.
Extra	Está no anexo B. É opcional.

SOBRE OS EXTRAS

Pular **todos** os extras não te deixa para trás em nada. Eles não são cobrados e nenhum capítulo depende deles. São para quem terminou antes e quer ir mais fundo. Um extra pode depender de outro extra, mas o conteúdo principal nunca depende de nenhum.

Sumário

01	O problema	5
02	Herança	8
03	Sobrescrever métodos	11
04	Polimorfismo	14
05	Fechamento	17
AN	Anexo A: erros que você vai ver	19
AN	Anexo B: extras	21
AN	Anexo C: glossário	23

CAPÍTULO 1

O problema

Antes de aprender coisa nova, você vai sentir falta dela. Este capítulo é um desafio sem ajuda. Faça primeiro, leia o capítulo 2 depois.

O desafio

O seu jogo tem uma classe `Personagem` só. Todo mundo é igual, muda só os números.

Acrescente dois personagens novos ao jogo:

- Um **Guerreiro**: mais vida e mais ataque que o normal, e o ataque dele nunca erra.
- Um **Mago**: pouca vida e pouca defesa, mas quando ele defende, também recupera 10 de vida.

Regra do desafio: use **só o que você já sabe**. Não pesquise, não pule para o capítulo 2.

CHECKPOINT 1

O Guerreiro e o Mago funcionam no jogo. O ataque do Guerreiro nunca erra. A defesa do Mago cura 10 de vida.

Não importa se o código ficou feio. Ficar feio é o resultado esperado.

Só continue depois de tentar

Existem dois caminhos para resolver isso com o que você já sabe, e os dois têm o mesmo defeito. Provavelmente você usou um deles.

CAMINHO 1: COPIAR A CLASSE INTEIRA

```

class Guerreiro:
    def __init__(self, nome, vida, ataque, defesa, pocoes):
        self.nome = nome
        self.vida = vida
        self.vida_maxima = vida
        self.ataque = ataque
        self.defesa = defesa
        self.pocoes = pocoes
        self.defendendo = False

    def mostrar_status(self):          # igual ao Personagem
        ...
    def esta_vivo(self):              # igual ao Personagem
        ...
    def receber_dano(self, quantidade): # igual ao Personagem
        ...
    def usar_pocao(self):             # igual ao Personagem
        ...
    def atacar(self, alvo):           # SÓ ESTE muda
        ...

```

CAMINHO 2: ENCHER A CLASSE DE IF

```

class Personagem:
    def __init__(self, nome, vida, ataque, defesa, pocoes, tipo):
        ...
        self.tipo = tipo

    def atacar(self, alvo):
        if self.tipo == "guerreiro":
            # nunca erra
            ...
        else:
            rolagem = random.randint(1, 20)
            ...

    def defender(self):
        if self.tipo == "mago":
            self.vida = self.vida + 10
            self.defendendo = True

```

Por que os dois são ruins

Responda para você mesmo, olhando o seu código:

1. Se você mudar a fórmula do dano, quantos lugares precisa editar no caminho 1?
2. Quantos `if` o método `atacar` vai ter quando o jogo tiver seis tipos de personagem?
3. No caminho 2, o Mago recebeu um atributo `tipo`. O Goblin também. O Goblin precisa dele?

O caminho 1 duplica código. O caminho 2 junta comportamentos diferentes dentro do mesmo método, e ele cresce sem parar.

Python tem uma ferramenta para exatamente este problema. É o próximo capítulo.

Terminou? Este capítulo não tem extras. Os extras começam no checkpoint 2, quando você já tiver herança na mão.

Herança

Herança é dizer que uma classe **é um tipo de** outra. O Guerreiro é um Personagem. Ele ganha tudo que o Personagem tem, de graça, e só escreve o que for diferente.

A sintaxe é o nome da classe-mãe entre parênteses:

```
class Guerreiro(Personagem):  
    ...
```

Lê-se: "Guerreiro é um Personagem".

Passo 1: o arquivo

Crie um arquivo novo chamado `classes.py`, na mesma pasta do `personagem.py`. Comece importando a classe-mãe:

```
from personagem import Personagem  
  
class Guerreiro(Personagem):  
    pass
```

O `pass` quer dizer "por enquanto, nada aqui dentro". Mesmo vazia, essa classe já funciona:

```
g = Guerreiro("Thoric", 120, 18, 8, 2)  
g.mostrar_status()
```

SAÍDA

```
Thoric  
Vida: 120 / 120  
Poções: 2
```

Você não escreveu `mostrar_status` no Guerreiro. Ele veio do Personagem. Isso é herança.

Passo 2: o construtor próprio

Do jeito acima, quem cria um Guerreiro ainda precisa digitar os cinco números. Mas todo Guerreiro tem os mesmos números. Quem deveria saber disso é a classe, não quem usa.

```
class Guerreiro(Personagem):
    def __init__(self, nome):
        super().__init__(nome, 120, 18, 8, 2)
```

Agora criar um Guerreiro é só:

```
g = Guerreiro("Thoric")
```

O que `super()` faz

`super()` é uma forma de chamar a classe-mãe de dentro da filha.

Quando você escreve `super().__init__(nome, 120, 18, 8, 2)`, você está dizendo: "*Personagem, monte o objeto do jeito que você já sabe, com estes valores*". O `__init__` do `Personagem` roda e cria `self.nome`, `self.vida`, `self.vida_maxima`, `self.defendendo` e todo o resto.

Sem essa linha, nada disso é criado. O objeto nasce vazio.

ORDEM IMPORTA

Chame `super().__init__(...)` na **primeira** linha do seu `__init__`. Só depois acrescente atributos novos. Se você inverter, o atributo novo pode ser apagado quando a classe-mãe montar o objeto.

Passo 3: o Mago

Faça o mesmo com o Mago. Ele é frágil: pouca vida, pouco ataque, pouca defesa, mas começa com mais poções.

```
class Mago(Personagem):
    def __init__(self, nome):
        super().__init__(nome, 70, 10, 3, 4)
```

CHECKPOINT 2

No arquivo `classes.py` existem as classes `Guerreiro` e `Mago`. As duas herdam de `Personagem`. As duas chamam `super().__init__()`.

Criar um deles precisa de **um** argumento só: o nome.

O `personagem.py` não foi alterado.

TESTE DO CHECKPOINT 2

Abra o `personagem.py` e acrescente esta linha no fim do `__init__` da classe

`Personagem`:

```
self.nivel = 1
```

Salve. Agora rode:

```
g = Guerreiro("Thoric")
print(g.nivel)
```

Resultado esperado: aparece `1` na tela.

Se der `AttributeError`: o seu `Guerreiro` não está chamando `super().__init__()`. Provavelmente você copiou as linhas do `__init__` do `Personagem` para dentro do `Guerreiro`, em vez de chamar a classe-mãe. Isso funciona por acaso, mas não é herança: o `Guerreiro` não recebe nada que você acrescentar no `Personagem` depois.

Depois do teste, pode apagar a linha `self.nivel = 1`.

LEIA A FONTE

Documentação oficial do Python, em português. Seção **9.5 Herança**:

docs.python.org/pt-br/3/tutorial/classes.html

A documentação mostra **outra** forma de chamar um método da classe-mãe, que não usa `super()`. Ache essa forma e escreva ela aqui:

Agora responda: se você renomear a classe `Personagem` para `Criatura`, qual das duas formas para de funcionar?

Terminou o checkpoint 2? Extras: anexo B, itens **E2.1** e **E2.2**.

CAPÍTULO 3

Sobrescrever métodos

Até agora as suas classes só mudam os números. O Guerreiro e o Mago lutam igual ao Personagem comum. Neste capítulo eles ganham comportamento próprio.

Sobrescrever é escrever, na classe filha, um método com o **mesmo nome** de um método da classe-mãe. Quando você chamar esse método no objeto da filha, a versão dela é que roda.

Existem duas situações, e elas se resolvem de formas diferentes.

SITUAÇÃO	O QUE FAZER	REGRA
Substituir	Não chama <code>super()</code>	O comportamento é <i>outro</i> . Você joga fora o da mãe.
Estender	Chama <code>super()</code>	O comportamento é o <i>mesmo</i> , <i>mais um pouco</i> .

Substituir: o ataque do Guerreiro

O ataque do Personagem erra quando a rolagem é menor que 5. O Guerreiro nunca erra. Isso é um comportamento diferente, não um acréscimo. Então você escreve o método inteiro e **não** chama `super()`.

Este é o método original, no `personagem.py`:

```
def atacar(self, alvo):
    rolagem = random.randint(1, 20)

    if rolagem < 5:
        print(self.nome, "errou o ataque")
        return

    dano = random.randint(self.ataque - 3, self.ataque + 3)
    alvo.receber_dano(dano)
    print(self.nome, "causou", dano, "de dano")
```

No `classes.py`, dentro da classe `Guerreiro`:

```
def atacar(self, alvo):
    dano = random.randint(self.ataque - 3, self.ataque + 3)
    alvo.receber_dano(dano)
    print(self.nome, "atacou sem falhar e causou", dano, "de dano")
```

Não tem a rolagem e não tem o `return`. O Guerreiro sempre acerta.

O `classes.py` agora usa `random`. Acrescente `import random` na primeira linha do arquivo, senão você vai ver `NameError: name 'random' is not defined`.

Estender: a defesa do Mago

O Mago defende igual a todo mundo, **e mais** recupera 10 de vida. A parte "igual a todo mundo" já existe. Seria desperdício reescrever.

Dentro da classe `Mago`:

```
def defender(self):
    super().defender()
    self.vida = min(self.vida + 10, self.vida_maxima)
    print(self.nome, "recuperou 10 de vida")
```

A primeira linha manda o Personagem fazer a parte dele, que é ligar o `self.defendendo`. Depois o Mago acrescenta a cura.

O `min(...)` impede a vida de passar do máximo.

COMO DECIDIR

Pergunte: "o que a classe-mãe faz ainda precisa acontecer?" Se sim, chame `super()`. Se não, não chame.

CHECKPOINT 3

- O `Guerreiro` sobrescreve `atacar` e nunca erra.
- O `Mago` sobrescreve `defender`, chama `super().defender()` e recupera 10 de vida.
- O `personagem.py` continua sem alteração.

TESTE DO CHECKPOINT 3

Rode isto:

```
m = Mago("Elric")
m.vida = 50
m.defender()
print("defendendo =", m.defendendo)
```

Resultado esperado:

```
Elric preparou a defesa
Elric recuperou 10 de vida
defendendo = True
```

Se aparecer `defendendo = False` e faltar a linha "Elric preparou a defesa": você esqueceu o `super().defender()`. O Mago está curando, mas não está defendendo de verdade. No jogo ele vai tomar dano cheio achando que se protegeu.

LEIA A FONTE

Mesma página, seção **9.5 Herança**:

docs.python.org/pt-br/3/tutorial/classes.html

A documentação diz que um método da classe filha que sobrescreve outro pode querer *estender* em vez de *substituir*. Copie a frase em que ela explica isso:

Terminou o checkpoint 3? Extras: anexo B, itens **E3.1**, **E3.2** e **E3.3**.

CAPÍTULO 4

Polimorfismo

Este capítulo tem pouco código novo. O que muda é o que você vai entender sobre o código que já existe.

Passo 1: os inimigos também viram classes

No `main.py`, os inimigos hoje são três Personagens com números diferentes:

```
inimigos = [  
    Personagem("Goblin", 40, 8, 2, 0),  
    Personagem("Orc", 70, 12, 4, 0),  
    Personagem("Troll", 100, 16, 6, 0),  
]
```

Transforme cada um em uma classe, no `classes.py`:

```
class Goblin(Personagem):  
    def __init__(self):  
        super().__init__("Goblin", 40, 8, 2, 0)  
  
class Orc(Personagem):  
    def __init__(self):  
        super().__init__("Orc", 70, 12, 4, 0)  
  
class Troll(Personagem):  
    def __init__(self):  
        super().__init__("Troll", 100, 16, 6, 0)  
  
    def atacar(self, alvo):  
        # o Troll bate com tudo, mas erra muito  
        rolagem = random.randint(1, 20)  
        if rolagem < 9:  
            print("O Troll errou feio")  
            return  
        dano = random.randint(self.ataque, self.ataque + 10)  
        alvo.receber_dano(dano)  
        print("O Troll acertou um golpe de", dano)
```

E no `main.py`:

```
inimigos = [Goblin(), Orc(), Troll()]
```

Passo 2: olhe o laço de combate

Agora vem a parte importante. Abra o `main.py` e olhe o laço:

```
for inimigo in inimigos:
    venceu = combate(jogador, inimigo)
    if not venceu:
        break
```

E dentro do `combate`:

```
if inimigo.esta_vivo():
    inimigo.atacar(jogador)
```

Você acabou de dar comportamentos diferentes para três classes diferentes. **Quantas linhas desse laço você precisou mudar?**

Nenhuma.

O laço escreve `inimigo.atacar(jogador)` e não sabe se o inimigo é Goblin, Orc ou Troll. Cada objeto sabe atacar do seu próprio jeito. Quem chama não precisa perguntar o tipo, não precisa de `if`, não precisa saber que o Troll existe.

Isso é polimorfismo. A mesma chamada, `inimigo.atacar(...)`, produz comportamentos diferentes dependendo do objeto que está ali.

Compare com o caminho 2 do capítulo 1, aquele cheio de `if self.tipo ==`. Lá, cada tipo novo obrigava a mexer no método. Aqui, cada tipo novo é uma classe nova e o resto do programa não fica sabendo.

CHECKPOINT 4

Goblin, Orc e Troll são classes que herdam de `Personagem`.

O `main.py` cria a lista com `[Goblin(), Orc(), Troll()]`.

O jogo roda do começo ao fim e o Troll ataca diferente dos outros.

Você não escreveu nenhum `if` novo dentro de `combate`.

TESTE DO CHECKPOINT 4

Este é o teste de verdade do polimorfismo. Crie mais um inimigo, no `classes.py`:

```
class Esqueleto(Personagem):
    def __init__(self):
        super().__init__("Esqueleto", 55, 11, 3, 0)

    def receber_dano(self, quantidade):
        print("Os ossos rangem")
        super().receber_dano(quantidade)
```

Agora acrescente `Esqueleto()` na lista de inimigos do `main.py` e jogue.

Resultado esperado: o Esqueleto entra na fila de inimigos, imprime "Os ossos rangem" toda vez que apanha, e o jogo funciona normalmente.

A pergunta do teste: fora a linha da lista, quantos arquivos e quantas funções você precisou modificar para o Esqueleto existir?

A resposta certa é **nenhuma**. Se você precisou mexer no `combate` ou no `turno_do_jogador`, alguma coisa não está herdando direito.

Terminou o checkpoint 4? Extras: anexo B, itens **E4.1**, **E4.2** e **E4.3**. O E4.3 é de leitura, não de código.

Fechamento

O que você construiu

Você começou com uma classe e um jogo em que todo mundo lutava igual. Agora tem uma família de classes:

```

Personagem
├── Guerreiro (ataca sem errar)
├── Mago      (defende e cura)
├── Goblin
├── Orc
└── Troll    (ataca forte e erra muito)
  
```

E o mais importante: o `personagem.py` está do jeito que estava no Dia 5. Você não precisou reescrever nada para estender tudo.

As três ideias, em uma frase cada

IDEIA	EM UMA FRASE
Herança	Uma classe é <i>um tipo de</i> outra e ganha tudo que ela tem.
<code>super()</code>	Chama a versão da classe-mãe, de dentro da filha.
Polimorfismo	A mesma chamada faz coisas diferentes conforme o objeto, e quem chama não precisa saber qual é.

Um cuidado antes da parte 2

HERANÇA É "É UM"

Só use herança quando a frase "X **é um** Y" fizer sentido. Guerreiro é um Personagem: pode. Poção não é um Personagem, mesmo que poção também tenha nome. Se a frase soar estranha, você quer um atributo, não uma classe filha.

E não empilhe. `Personagem` → `Guerreiro` é o bastante. Se você criar `Personagem` → `Humano` → `Guerreiro` → `GuerreiroDeElite`, daqui a duas semanas ninguém acha mais onde um método foi definido.

O que vem na parte 2

- **Encapsulamento:** por que `inimigo.vida = -999` devia ser proibido, e como o objeto se protege.
- **Seleção de personagem:** o jogador escolhe entre Guerreiro e Mago no início. Isso é polimorfismo virando tela de menu.
- **Chefe final** com mecânica própria.
- O jogo fechado, do início ao fim.

ANEXO A

Erros que você vai ver

Estes cinco erros aparecem para quase todo mundo. Ver o erro não quer dizer que você errou feio: quer dizer que chegou na parte difícil. Ache a mensagem aqui e siga a instrução.

1. OBJECT HAS NO ATTRIBUTE

```
AttributeError: 'Guerreiro' object has no attribute 'nome'
```

O objeto foi criado sem passar pelo `__init__` do `Personagem`. Quase sempre é um `__init__` na filha que não chama `super().__init__(...)`.

ERRADO

```
class Guerreiro(Personagem):
    def __init__(self, nome):
        pass                # nao monta nada
```

CERTO

```
class Guerreiro(Personagem):
    def __init__(self, nome):
        super().__init__(nome, 120, 18, 8, 2)
```

2. TAKES 2 POSITIONAL ARGUMENTS BUT 6 WERE GIVEN

```
TypeError: Guerreiro.__init__() takes 2 positional arguments but 6 were given
```

O seu `Guerreiro.__init__` pede só o nome, mas você criou o objeto passando cinco valores, como fazia antes.

```
g = Guerreiro("Thoric", 120, 18, 8, 2)
```

```
g = Guerreiro("Thoric")
```

Os números agora ficam dentro da classe. É esse o ganho.

3. MISSING REQUIRED POSITIONAL ARGUMENTS

```
TypeError: Personagem.__init__() missing 4 required positional arguments:  
'vida', 'ataque', 'defesa', and 'pocoas'
```

O contrário do anterior. A sua classe filha ainda não tem `__init__` próprio, então ela usa o do `Personagem`, que exige os cinco valores.

Ou você escreve o `__init__` da filha, ou passa os cinco na hora de criar.

4. NAME 'PERSONAGEM' IS NOT DEFINED

```
NameError: name 'Personagem' is not defined
```

Faltou o import no topo do `classes.py`:

```
from personagem import Personagem
```

Se a mensagem falar de `random`, falta `import random` no mesmo lugar.

5. O MÉTODO EXISTE MAS NÃO RODA

Sem erro nenhum, e mesmo assim o Guerreiro continua errando o ataque. Duas causas:

- **Indentação.** O `def atacar` ficou fora da classe. Ele precisa estar recuado quatro espaços, alinhado com o `def __init__`.
- **Nome diferente.** `Atacar` ou `atacaR` não sobrescrevem `atacar`. Python diferencia maiúscula de minúscula, então você criou um método novo em vez de substituir o antigo.

Extras

Tudo aqui é opcional. Nenhum capítulo depende destes itens, e pular todos não te atrapalha em nada na parte 2.

Um extra pode depender de outro extra. Quando depender, está escrito embaixo do título.

E2.1 O ARQUEIRO

Depende de: checkpoint 2.

Crie a classe `Arqueiro`, herdando de `Personagem`. Vida média, ataque médio, defesa baixa, e comece com 3 poções. Sem sobrescrever método nenhum, só o `__init__`.

Teste criando um e chamando `mostrar_status()`.

E2.2 UM INIMIGO SEU

Depende de: checkpoint 2.

Invente um inimigo com nome e números escolhidos por você. Coloque ele na lista do `main.py` e jogue até derrotá-lo.

E3.1 O ASSASSINO E O ACERTO CRÍTICO

Depende de: checkpoint 3.

Crie a classe `Assassino`. Ele sobrescreve `atacar`: rola um número de 1 a 20 e, se tirar 18 ou mais, o dano é **dobrado** e ele imprime "ACERTO CRÍTICO".

Pense antes de escrever: isso é substituir ou estender? Você precisa do `super()` aqui?

E3.2 O PALADINO

Depende de: checkpoint 3.

Crie a classe `Paladino`. Ele estende `usar_pocao`: usa a poção normalmente e ainda recupera 5 de vida a mais.

Cuidado: o `usar_pocao` do `Personagem` às vezes desiste no meio, com `return`, quando não há poções ou a vida está cheia. Rode e veja o que acontece com o seu bônus nesses casos. Isso é um bom problema para pensar.

E3.3 O MENU QUE NÃO DESPERDIÇA O TURNO

Depende de: checkpoint 3.

No `main.py`, digite uma opção inválida durante o combate. Repare que você perde o turno e o inimigo ataca mesmo assim.

Conserte: o `turno_do_jogador` deve continuar perguntando até receber 1, 2 ou 3.

E4.1 O CHEFE QUE FICA FURIOSO

Depende de: checkpoint 4 e do extra E2.2.

Crie um chefe que sobrescreve `receber_dano`. Quando a vida dele cair abaixo da metade, ele imprime "O chefe entrou em fúria" e o ataque dele aumenta em 5, uma vez só.

Coloque ele no fim da lista de inimigos, como último desafio.

E4.2 A ARENA

Depende de: checkpoint 4 e dos extras E2.1, E3.1 e E3.2.

Escreva um arquivo `arena.py` que coloca todas as suas classes numa lista e faz cada uma lutar contra as outras, sem ninguém digitar nada. O computador escolhe as ações sozinho.

No fim, imprima quem sobreviveu a mais lutas.

Se você fez tudo certo até aqui, a arena não precisa de um único `if` perguntando o tipo do personagem.

E4.3 LEITURA DIFÍCIL

Depende de: checkpoint 4.

Este extra é de leitura, não de código. Abra a referência da biblioteca padrão, na entrada da função `super`:

docs.python.org/pt-br/3/library/functions.html#super

É um texto denso, escrito para quem já programa. Fala em objeto proxy, MRO e herança múltipla cooperativa. Você não precisa entender tudo.

Responda duas coisas:

1. O que `super()` devolve? Não é a classe-mãe. É o quê?
2. A página mostra `super()` sem argumentos e `super(C, self)`. Segundo o texto, quando as duas formas são equivalentes?

Glossário

Os termos que apareceram na teoria, com o código deste material do lado.

TERMO	O QUE É
Classe	O molde. <code>Personagem</code> é uma classe.
Objeto	Uma coisa feita a partir do molde. <code>Guerreiro("Thoric")</code> produz um objeto.
Instância	Outro nome para objeto. "Thoric é uma instância de Guerreiro."
Instanciar	Criar o objeto a partir da classe. É o que acontece quando você escreve <code>Guerreiro("Thoric")</code> .
Atributo	Um dado guardado no objeto. <code>self.vida</code> , <code>self.nome</code> .
Método	Uma função que pertence a uma classe. <code>atacar</code> , <code>defender</code> .
Função	Um bloco de código com nome, que não pertence a nenhuma classe. <code>combate(...)</code> no <code>main.py</code> .
Construtor	O <code>__init__</code> . Roda sozinho quando o objeto é criado.
<code>self</code>	Dentro de um método, é o próprio objeto em que o método foi chamado.
Classe-mãe	A classe de quem se herda. Também chamada de superclasse ou classe base. <code>Personagem</code> .
Classe filha	A que herda. Também chamada de subclasse ou classe derivada. <code>Guerreiro</code> .
Herança	Uma classe ganhar os atributos e métodos de outra.
Sobrescrever	A filha escrever um método com o mesmo nome de um da mãe, para mudar o comportamento.
<code>super()</code>	Chamar a versão da classe-mãe de dentro da filha.
Polimorfismo	A mesma chamada de método produzir comportamentos diferentes conforme o objeto.
Encapsulamento	O objeto proteger o próprio estado, deixando que mudanças passem só pelos métodos dele. É o tema da parte 2.